

14.1 Threads and Processes

In the execution of a program, we can effectively visualise the flow of control by imagining a *thread* which, in the textual structure of the program, links all the commands reached by the control according to the order given by the execution. A control thread, or simply thread, identifies the sequence of commands of a program executed in a specific computation. In a *sequential* program only one thread is active at any time of execution, while in a *concurrent* program there are several active threads. Often, instead of threads, people talk about processes, tasks or concurrent programs, although they consider the case where there are several contexts, of various granularity, active at the same time. Also at the level of languages, the terminology is not consistent, since some languages call processes what we have called threads, while others (Ada for example) call them tasks.

Here, we will try to use a uniform terminology, and we will use the term thread with the meaning seen above, while we will use the term *process* to indicate a generic set of executing instructions, with its own address space, as it usually happens in operating systems.¹ According to this view, we can see a process as consisting of several threads sharing resources (e.g., memory).² Typically, operating systems distinguish between *heavyweight* processes, which have their own address space, from *lightweight*, which may share a common space. This terminology is sometimes also used for threads; here we always consider lightweight threads.

¹ Note that, in the field of concurrency theory and the so-called “process algebras”, the term “process” indicates an abstract computational entity, often identifiable with a real program.

² Nonetheless, from the implementation standpoint, one can realise threads using primitives of the kernel of the operating system to manage processes.

14.2 A Brief Historical Overview

Like many other important areas of computer science, concurrency arose from an innovation in hardware architecture: the introduction, in the 1960s, of *device controllers*. These are hardware mechanisms that allow peripheral devices to autonomously manage the input and output of data, without the direct intervention of the CPU. Before the advent of these devices, the management of peripherals was extremely inefficient: the CPU would send a command to the peripheral device to perform an input or output operation and then wait for the command to complete. Since peripherals were (and are) devices with relatively longer operating times (than, e.g., memory registries), accessing peripherals meant a sensible amount of CPU cycles wasted waiting for the peripheral to provide/emit some data (just think that a card reader could read about four cards per second, while a computer already in the sixties could execute about forty thousand instructions per second). Device controllers and their communication mechanisms, based on *interrupts*, solved this inefficiency: now, when the CPU wants to perform an input or output operation, it sends a command to the appropriate device controller and then continues with the execution of other instructions. As soon as the peripheral has finished executing, the controller signals the CPU with an interrupt. The signal tells the CPU that the command it sent to the peripheral terminated and there is some data available for it to process. Usually (when possible), when the CPU receives these signals, it interrupts the execution of the current instructions, and it deals with the signal through its associated interrupt handler (which e.g., instructs the CPU on what to do with the data read from the peripheral).

While interrupts made CPU-peripheral interactions concurrent and more efficient, they introduced new problems in programming, which, in turn, compelled researchers to find mechanisms to manage concurrency. In particular, concurrency can cause *race condition*, i.e., situations that occur in the presence of access to shared data, when the result of the computation may depend on the relative order of the operations. As an example, imagine a program that performs some calculations using some data from the disk. We would like our program to ask the peripheral to read the data and, while this happens, it should continue to compute other values. However, we cannot predict when the signal from the disk interrupts the computation: the interrupt could stop the calculation after it loaded some values in memory, the interrupt handler could overwrite those values, and, at the resumption, the program could operate on the wrong set of data. In general, concurrency introduces the possibility for the CPU to execute concurrent instructions in combinations of orderings that may result in different execution results. To better see this, let us consider a simpler example (although equivalent in meaning) of the CPU-peripheral scenario we drew above. In our simplified example, we have two threads $T1$ and $T2$ in a program and they both want to increment the value of a shared variable x , first copying it from the central memory to a registry. Without additional constraints on the ordering of execution of $T1$ and $T2$, a valid execution is (prefixing each action with the thread that executes it).

```
T1 | read x in reg1
T2 | read x in reg2
T2 | increment reg2
T2 | write reg2 in x
T1 | increment reg1
T1 | write reg1 in x
```

What happened above is that $T1$ sort of nullified the effect of $T2$ on x . Let us say that x contains the value 0. $T1$ reads the value of x , which is 0. Then, also $T2$ reads the value 0 of x and it continues by incrementing it to 1 and writing the increment in x . Finally, $T1$ increments its reading of x to 1 and writes it in x . Of course, if we had, e.g., $T1$ and $T2$ execute one after the other, x would hold the value 2, instead of 1. Notably, the scenarios above are just two out of 20 admissible sequencing of the instructions of $T1$ and $T2$, of which only two lead to x holding the value 2—the cases in which $T1$ or $T2$ execute completely, followed by the complete execution of the other thread.

As seen, reasoning on the behaviour of even small concurrent programs is hard, due to the number of paths that the computation can follow. One way to make concurrent computations predictable is to introduce synchronisations among the various operations. Indeed, the introduction and adoption of interrupts coincided with the introduction of the first tools for synchronisation and concurrency management, such as critical sections and mechanisms for mutual exclusion. The first multiprogramming systems, of the batch type, allowed very limited concurrency, essentially only at the level of the operating system (one process ceded access to the CPU to another one only on input and output operations). Subsequently, with the advent of time-sharing systems, concurrent execution also appeared at the application level, followed by the inclusion of constructs for communication and synchronisation in high-level languages.

A second important impulse to the development of techniques and models for concurrency came, at the end of the sixties, from the realisation of the first multiprocessor architectures. The presence of several processors in these architectures supports the simultaneous (or parallel) execution of different programs or even different parts of the same program. This possibility posed new problems, both algorithmic and methodological; in fact, *parallel programming*, developed mainly for scientific computing applications, also involves fields other than those usually connected with programming languages.

A third important element for the development of concurrent programming came in the 1970s with the first computer networks. The possibility to make different machines, even far apart, “talk” to each other opened the way to another, different model of concurrency: *distributed computing*, in which programs that run on different machines can cooperate using mechanisms based on the exchange of messages implemented by a suitable communication infrastructure.

14.3 Types of Concurrent Programming

From the brief historical note above, it appears that we can distinguish various types of concurrent programming.

For starters, we can distinguish *physical concurrency* from *logical concurrency*. In the first case, we have simultaneous, or parallel, execution of two threads (or processes, or even programs), as, e.g., in the case of machines with several processors. In the second case, we have concurrency only at the logical level, i.e., the programmer and the software application perceive execution as concurrent but, physically, the concurrent activities map into sequential executions. This is the case of users that simultaneously use a machine with a single processor in multiprogramming. Conceptually and through the lenses of programming languages, this distinction does not have much relevance, except when considering system performance and execution time—broadly speaking, if we have a sequential solution to a problem that runs in T_s time units and a concurrent solution to the same problem that runs in T_p , with physical concurrency we would have $T_p < T_s$.³

Parallel Programming

The type of programming that uses physical concurrency is usually referred as *parallel programming* and it involves dedicated algorithmic and methodological aspects necessary to define an efficient parallel solution to a problem. In this field, we can distinguish between *data parallelism*, where the main source of parallelism derives from the application of the same operation to different data, and *operation parallelism*, where we apply different operations in parallel to the same set of data. Both types of parallelism have important practical applications, especially in the field of scientific computing, which employs specific multiprocessor architectures such as vector processors. The typical applications of parallel programming concern the solution of differential equations, such as those of weather models, where iterative methods based on matrix calculations can be effectively parallelised to solve systems of equations. The division of the work between the various processors leads to significant increases in performance, provided that the algorithms and programming methods used in the solution support load-balancing among the processors.

Although parallel programming introduces specific algorithmic and methodological problems, from a linguistic point of view it does not pose problems particularly different from those of logical concurrency. Indeed, a parallel program can use various underlying architectures, from multiprocessor machines with shared memory

³ How much less depends on many factors. In general, the performance increase, also called *speed-up*, is given by the quantity T_s/T_p : if this is equal to n , we have a linear speed-up (also called perfect), but the speed-up is frequently strictly less than n . There are also cases in which the speed-up is greater than n (or superlinear): this happens, for example, if the computation needs a cache much bigger than the one of a single processor, so that sequential performances are particularly penalized with respect to concurrent ones.

to multicomputers⁴ or networks of computers with separate memory. For communication and synchronisation, one such program can use the same basic mechanisms (shared memory and message exchange) that we see in the context of logical concurrency and for this reason, in the following, we focus mainly on the latter.

In the context of *logical concurrency*, hereafter referred to simply as concurrency, we distinguish between *multithreaded programming* and *distributed programming*.

Multithreaded Programming

We introduced the notion of multithreaded programming above and it concerns the presence of more than one thread or process. Specifically, it entails more than one context of execution active at the same time, in an application that runs on a single machine—either a single processor or a shared-memory multiprocessor. The necessity and usefulness of using concurrent threads and processes depends on the nature of the problem the application has to solve. Let's consider, for example, a modern browser: when we follow a link to open a new page (and often a new window) the program can open a new thread (or even a new process) whose tasks are to manage the communication of packets related to the new page, to manage its formatting and rendering (possibly opening new threads) and more. In the meantime, the user can interact with other functions of the browser, without the risk that the execution of slow operations (such as displaying images) could penalise the faster ones. Of course, in single-processor machines, we need some implementation techniques to avoid threads that idly occupy the CPU (for instance, waiting for the completion of a slow operation) or for too long (preventing others to continue their computation).

Distributed Programming

The term distributed programming refers to the creation of concurrent programs designed to run on machines made of distributed physical structures. These can be multi-computers with distributed memory but also actual networks of computers of various architectures and topologies. The important point, here, is that the nodes of the computation system are physically distributed, they cannot share their memory, and we must introduce appropriate mechanisms and communication primitives based on message exchange. Chapter 15 is dedicated to deepening the discussion on this last concurrency modality and its recent developments under the scope of languages.

Other Types of Concurrent Programming

A second axis along which we can distinguish various aspects of concurrent programming concerns the granularity of the concurrent activities: these may be low-level instructions, high-level language commands, parts of programs or entirely separate programs.

⁴For the sake of clarity, we use the term “multiprocessor” for machines with shared memory and “multicomputer” for machines with distributed memory. The terminology in this field is not univocal, and there are many possible variations on the theme. For the present discussion, what is important is the presence or absence of shared memory between different processors.

The two extreme cases of this hierarchy are not particularly interesting for the purposes of the present discussion, since they concern architectural issues and aspects related to communication protocols rather than specific linguistic aspects. Also single-command concurrency, although important in practice, is more concerned with efficient parallel compilation than with explicit concurrency management constructs. Indeed, in this type of concurrency we usually find a traditional sequential language extended with special annotations that inform the compiler on how to distribute data in memory, on how many processors to use and so on. The compiler, using this information, produces an optimised parallel object code that can take advantage of the features of the physical machine (typically multiprocessors with shared memory). An example of a language that allows this kind of parallel compilation is High Performance FORTRAN (HPF), which is an extension of the FORTRAN 90 language, proposed in the nineties, which allows obtaining programs with parallelism on data and that is particularly used for scientific applications.

In this chapter, therefore, we focus on concurrency at the level of parts of programs, be they subprograms, units, etc. Indeed, in this case, we need explicit high-level linguistic constructs to manage the communication and synchronisation of the parts, as we see in more detail in the next section.

A Note on Terminology

The following discussion, except for the specific case of Java, dealt with in Sect. 14.7, makes no particular distinction between the case of threads and that of processes. Hence, for the sake of simplicity and until Sect. 14.7, we use the term “process” as a synonym of “thread”.

14.3.1 Communication Mechanisms

In the context of concurrent programming (regardless of the type), two common aspects of different models and languages are the mechanisms of *communication* and *synchronisation*.

The necessity of communication mechanisms is quite obvious: we need a way to let the different activities that contribute to the realisation of some function communicate with each other, to exchange information of various kinds (for example, partial results of computations or control communications). In the mechanisms that consider two communicating partners, we can identify two roles: the one who produces some information and the one who receives it.⁵ Conceptually, we can identify two primary communication mechanisms, depending on how the producer and receiver of the communication are realised: those based on *shared memory* and those using *message exchange*. These two types actually identify different classes of concurrency models that we analyse in detail in Sects. 14.4 and 14.5. Here we are only interested in mentioning their basic characteristics.

⁵ In the field of communication theory, these two partners are usually called sender and receiver; however here, for uniformity with current terminology, we reserve these terms only for the model with message exchange.

Blackboard

A communication model somewhat intermediate between shared memory and message exchange is the one based on the *blackboard* architecture (or shared store). There, separate processes, which may have their own private memory space, share a memory area, called blackboard (or store), and use it for communication. The best-known model in this field is the Linda language model, where the blackboard is dubbed *tuple space* and it is an abstraction of an associative memory shared among the processes. The processes communicate with each other using appropriate communication primitives to send/receive messages to/from the tuple space. Hence, even if it uses message exchange, the communication is indirect and mediated by shared memory. This model, studied in the context of the so-called “coordination languages”, although interesting, is less relevant than the other two and omitted in this text.

Communication with Shared Memory

This kind of communication is what we have already seen in other parts of this book and is typical of the imperative paradigm: two parts of the same program, that have access to the same shared variable, can communicate simply by writing to the variable and reading its value. More generally, in the shared-memory concurrent programming model, the two partners, for example, two different processes, have access to the same memory space where they can write and read values. This memory space can be shared with more than one partner, thus allowing broadcast communications where the same information produced is received by more than one partner.

Communication with Message Exchange

According to this communication model, the partners do not share any memory space: to communicate, the sender must explicitly perform a `send` operation, while, in a dual way, the receiver must perform an explicit `receive` operation. Moreover, both the sender and the receiver should be able to access an appropriate communication structure, usually called a *channel*, to provide a path that, at least from the logical point of view, connects the two partners. The implementation of this structure, as well as that of send and receive operations, can be the most varied and can depend both on the abstraction level of the communication operation and the underlying hardware structure. It is worth remembering that, in principle, the communication mechanisms of the physical machine and those of the language implemented on it can be different. For example, it is possible that a program that was written in a language that implements communication with message exchange runs on a multiprocessor machine that uses shared memory to simulate the channels and realise send and receive operations. Similarly, it is possible, at least in principle, to run programs written in a language that uses shared-memory communication on a multicomputer with distributed memory and hardware-level communication based on message exchange.

14.3.2 Synchronisation Mechanisms

The second fundamental aspect of the existing models of concurrency concerns the employed *synchronisation* mechanisms. These mechanisms allow one to control the relative order of the activities in the different processes and they are essential to ensure the correctness of the computation. In the context of shared memory communication, this aspect is evident, and synchronisation mechanisms are present in an explicit way. Let us consider, for example, the case of two processes that communicate using a shared variable x : of course, the receiving process must retrieve the value of x only after the producing process has written the correct value in x . More in general, in concurrent programming with shared memory, we find two types of synchronisation: *mutual exclusion*, which guarantees that certain critical regions of the code are not accessible to several processes at the same time, and *condition synchronisation*, which allows suspending the execution of a process until an appropriate condition occurs.

These two types of synchronisation are realised using *busy waiting* (also known as *spinning*) and the *block* (also known as scheduler-based synchronisation). In the former, the waiting process actively uses the CPU by executing a cycle in which it continuously tests a condition. In the second one, the process that has to wait releases the CPU, so that it can be used by others. The scheduler will then reactivate the process when the waiting condition will be verified. Various mechanisms allow this kind of synchronisation, in particular semaphores and monitors. We see all these techniques in detail in Sect. 14.4.

Synchronisation mechanisms are also present in the context of message exchange communication, however, these are usually implicit in the definition of the *send* and *receive* primitives. Indeed, before receiving a message, the latter must have been sent. The *receive* primitive may directly implement this synchronisation, for example, by suspending the activity of the process until the message has been sent, without the need for any further explicit construct. Considering the type of synchronisation performed in the *send* and *receive* primitives, we can identify two communication mechanisms: a *synchronous* and an *asynchronous* one, which we discuss below.

Asynchronous Communication

In this type of communication, the sending and receiving of a message take place at different times, as in the case of e-mail. The sender performs a *send* operation and then proceeds with the computation without waiting for the recipient to perform the corresponding *receive* operation. In this case, the *send* operation is said to be non-blocking. The *receive* operation, on the other hand, is blocking: the recipient process is blocked after performing a *receive* and waits for the sender to perform the *send* operation—the receiver can continue the computation only after this has been done. Asynchronous communication presents some difficulties in the implementation of channels, since they can, in principle, contain an arbitrary number of messages sent and not yet received.

Synchronous Communication

In the case of synchronous communication, this happens only when both the sender and the receiver are ready to communicate using a specific channel, as with a telephone call (without the use of an answering machine). Conceptually, in synchronous communication, the `send` and the `receive` are done at the same time. In practice, the operations occur at different times, meaning that the sender, after having performed a `send` operation, can continue with its computation only after the recipient performed the complementary `receive` operation. The recipient behaves in the same way as in the case of asynchronous communication. In the case of synchronous communication, both the `send` and the `receive` are blocking. The implementation of synchronous communication may use a limited-size buffer to realise the communication channel, since a new message cannot be sent until the previous one has been received.

We would like to draw the reader's attention not to misunderstand the above terminology, which refers to communication mechanisms and not to execution mechanisms. If we consider the latter, the languages that we discuss in this chapter are all asynchronous, in the sense that each process, thread, task or program is executed at its own speed (of course, depending also on the particular underlying processor) without this being directly linked to that of other processes. Indeed, there exist synchronous languages and models, where the execution times of different processes can be synchronised—either because of the type of architecture used or because of the assumptions made on the abstract model of computation employed. These languages are of considerable interest in real-time and embedded systems, but their discussion is beyond the scope of this chapter.

14.4 Shared Memory

In this section, we examine the synchronisation mechanisms used in concurrent programming models based on shared memory. As mentioned, this mechanism supports communication between two or more partners by letting them access the same shared memory area which, for simplicity, we identify as a variable. For the communication to be correct, we need specific synchronisation techniques to avoid race conditions and other incorrect situations. As seen in Sect. 14.3.1, the two main forms of synchronisation for shared memory are mutual exclusion and condition synchronisation. *Mutual exclusion* allows multiple processes to execute simultaneously the sections of code that access shared resources—the so-called *critical sections*. In this way, entire sections of code can be considered atomic in terms of execution. *Condition synchronisation*, on the other hand, allows one to suspend the execution of a process until an

appropriate condition occurs, such as a variable that assumes a certain value.⁶ These two types of synchronisation are achieved using two techniques: *busy waiting* (also known as spinning) and *block* (also called scheduler-based synchronisation).

14.4.1 Busy Waiting

In busy waiting, the process that has to wait its turn (to enter a critical section or a given condition has occurred) executes a cycle in which it continuously evaluates a condition until it becomes true. During the wait, the CPU is actively employed, hence, using this technique on a uniprocessor machine to synchronise different processes does not make much sense, since it is wasteful to occupy the CPU idly waiting for a condition to occur (e.g., emptying a buffer) that results from another process using the same CPU. On the other hand, it is an efficient technique if used on multiprocessor machines (or even if used at the hardware level, for example, to synchronise data transfer on buses and local networks). Various synchronisation mechanisms use active waiting: the most important is the *lock* (to realise mutual exclusion) and the *barrier* (to realise condition synchronisation). Let us see both of them.

Locks

First, to realize a mutual exclusion mechanism, we need to make it clear which atomic instructions are available to us, that is, which are the instructions whose indivisibility is guaranteed by the underlying machine. Usually, at the hardware level, there is an atomic instruction `test_and_set( B )` that returns the value of the Boolean variable `B` and then, atomically, i.e., in a non-interruptible way, it assigns the value `true` to that variable. Using such instruction we can realize mutual exclusion as follows.

Suppose that each process that wants to access the critical section has the following structure (as usual, we use our pseudo-language quite freely here):

```
process Pi {
    non-critical section;
    acquire_lock( B );
    critical section;
    release_lock( B );
    non-critical section;
}
```

The critical section, as already mentioned, encloses the shared data structures for which we want to guarantee mutual exclusion, that is, for which we want to prevent concurrent accesses. Let us also assume that the shared variable `B`, of type Boolean, is initialized to `false`:

```
bool B = false;
```

⁶ Mutual exclusion might seem a special case of condition synchronisation where the condition is: “no other process is in a critical section”. This, however, would assume control over the global state of the system, which is not usually the case in condition synchronisation.

This variable realizes the so-called lock, that is, the lock to be used to access the critical section. Thus, using the instruction `test_and_set( B )`, we can define the lock acquisition and release functions as follows:

```
void acquire_lock( ref B: bool ) {
    while test_and_set( B ) do { skip; }
}

void release_lock( ref B: bool ) {
    B = false;
}
```

(`skip`, above, is the command that produces no effect, included in the example only for clarity of exposition). The procedure for lock acquisition consists of a loop (also called *spin*, hence the term *spin lock* for this technique) where we test the shared variable `B` until it has value *false*, denoting the possibility to access the critical section. The test is done in the same atomic action that assigns to `B` the value *true*, such that only one among several tests tried simultaneously (by multiple processes) is successful. This implies that only one among the several processes executing `acquire_lock( B )` can succeed in accessing the critical section, thanks to the atomicity of the operation of `test_and_set`, which guarantees mutual exclusion. Note that this mechanism also ensures that one, out of the many processes that are trying to enter the critical section, succeeds in doing so, i.e., there is always at least one process that is able to progress (the one entering the critical section). In other words, with this configuration, we can guarantee the absence of deadlocks. Unfortunately, the previous implementation of mutual exclusion presents a serious efficiency problem on multiprocessor machines. The processes attempting to access the critical section, keep performing write operations to the shared variable each time they execute the `test_and_set` operation within the while loop. Such operations, performed atomically along with testing the value of the variable, cause contention for memory and bus usage that eventually degrades system performance.

To avoid this problem, one can resort to the *test and test_and_set* technique. Instead of performing a `test_and_set` operation on the shared variable, resulting in a write, we only perform the test (read) operation in each cycle. When that operation is successful, we then switch to the `test_and_set` to actually gain access to the critical section. According to this technique, the lock acquisition function is realized as follows (release occurs as before):

```
void acquire_lock( ref B: bool ) {
    while B do { skip; }
    while test_and_set( B ) do {
        while B do { skip; }
    }
}
```

Both this and the previous implementation of mutual exclusion present an additional problem. There is no guarantee that a process that wants to access the critical section sooner or later will succeed in doing so. Indeed, it may be the case that, when the critical section is released, there is always another process that manages to access it first. This is because we make no assumptions about the management of processes and, in particular, their scheduler. Although, in practice, such behaviour seems quite

unlikely, there are specific algorithms that ensure that a process waiting to access the critical section, or more generally waiting to proceed, will sooner or later succeed in doing so. These properties are called “fairness” and their precise definition is beyond the scope of our treatment (there are different notions of fairness, and in some cases the differences are subtle). Here, we settle for an intuitive use of this term: assuming that there are multiple claimants for a common resource, a policy for managing access to the resource is (weakly) *fair* if every claimant who is enabled to access it infinitely many times succeeds in accessing the resource. We can achieve a fair solution for mutual exclusion by following the idea that, in the case where multiple processes are competing for access to the critical section, we can establish shifts among these. An algorithm that implements this idea in an intuitively simple way is that of ticket machines—the ones found in many places to manage customer access to a counter. We give a dispenser of numbers (tickets), sorted in ascending order, to customers, and we grant access to the resource, in mutual exclusion, following that order. A limitation of this algorithm is that the values grow in an unlimited way, at least in principle. Specialised literature includes examples of algorithms that can avoid this unlimited growth, like the tie-breaker.

Barriers

Barriers are a condition synchronisation mechanism typically used in parallel programming, where we often have algorithms implemented by n different processes that concur to produce the final solution using iterative methods (e.g., matrix manipulation). At the end of each iteration, each process must wait for the completion of the activities of all other processes before moving on to the next iteration. This can be achieved by introducing a synchronisation point in the code, called a *barrier*, where each process waits for a global condition to occur before proceeding. Each process P_i , with $i = 1 \dots n$, will therefore have a pattern of this type

```
process Pi {
  while B true do {
    code of task i;
    wait for all other Pj to terminate;
    // barrier
  }
}
```

To realise the wait, and thus the barrier, we can use a global counter, initialised at zero before the cycle, which each process increments at the end of its task and on which it expects the value n to be reached. The access operations to the counter must be done atomically and the waiting can be active.

There are many ways, even quite sophisticated, of realising barriers. As they are quite specific to parallel programming, we do not consider them further here.

14.4.2 Scheduler-Based Synchronisation

The active waiting mechanisms seen in the previous section have several drawbacks. First, there are normally more processes than processors, so it is not convenient to use CPU time to perform an active wait for one process, while other processes are waiting to use the CPU.⁷

Furthermore, the variables used for synchronisation in our examples, as well as in practice, are “normal” variables and not structured in a specific data type or in some linguistic construct for this purpose. As we have learned from our discussion on data types and objects, if we want to minimise the probability of making programming errors, we should strive to keep concerns separated, e.g., syntactically or via the type system.

Scheduler-based synchronisation mechanisms, also called *block-based* mechanisms, have been introduced to deal with the above problem. In scheduler-based synchronisation, when we put on hold a process, we associate the event that the process is waiting for with information that identifies the process itself. When the event occurs, the scheduler knows that the processes associated with that event are ready for execution. These synchronisation mechanisms usually provide variables that can take specific values and support specific operations on those values for synchronisation. Thus, we can see them, at least at the conceptual level, as data types, although they rarely appear explicitly as constructs in high-level languages.

In the following, we look at the two main mechanisms of this type, i.e., semaphores and monitors, which support both mutual exclusion and condition synchronisation.

Semaphores

The mechanism based on *semaphores*, introduced by Dijkstra in the 1960s, was the first explicit synchronisation tool for shared-memory systems and remains to this day one of the most important synchronisation constructs implemented at the core operating system level. Already present in ALGOL 68, semaphores are used in almost all libraries for concurrent programming and also appear in some languages, such as SR and Modula-3.

As its name suggests, we can see a semaphore as a construct that regulates the “traffic” of processes to avoid “collisions”. This is similar to the namesake device that regulates traffic on railway lines, where a colour of a dedicated semaphore indicates whether a track section is free, and therefore allows or forbids access to trains. The colour of a semaphore is manipulated by specific operations when a train occupies (from green to red) or releases (from red to green) the track section.⁸

⁷ An exception to this principle occurs when the active wait time is less than the time required to make the context switch needed to execute another process.

⁸ Here, we consider the simplified, binary version of the train being or not in the track section and avoid the complexity of having a third colour, usually yellow, that indicates to other trains that another train is leaving/entering the section.

The metaphor translates into semaphores being the data type whose values (corresponding to the colours) determine the access to sections of the program by processes. Our semaphore data type, which we call `sem`, is made of:

- the set of values, consisting of the integers ≥ 0 ;
- two atomic operations called `P` and `V`.⁹

A variable `s` of type semaphore, declared as usual with

```
sem s
```

provides a shared structure that supports synchronisation between processes based on the value of `s`, by means of the two operations `P` and `V` where, intuitively, `P` gives access to the semaphore and `V` releases the semaphore.

More in detail, if a process `Pr` performs the operation

```
P ( s )
```

the effect is as follows. If the value of `s` is > 0 , then `s` is decremented and the operation terminates, so the process can continue with the execution of the code following `P ( s )` (i.e., accessing the critical section). Alternatively, if the variable `s` has value 0, then the operation itself (and process `Pr` by extension) blocks until `s` takes value > 0 . This blocking is usually handled at the level of the operating system kernel. This means that the process `Pr` put on hold releases the CPU, which others can use, and the operating system “awakens” it with the signalling of the event that the variable `s` has a value > 0 .¹⁰ As mentioned, these operations using `P ( s )` must be thought of as being performed in a single atomic action. This means that, for example, if the value of `s` is 1 and two processes are performing an operation `P ( s )`, only one of the operations executes, entering the critical section, while the other blocks.

The operation

```
V ( s )
```

allows one to signal that the resource that had been committed to a `P ( s )` is now free. To do this, `V ( s )` increments, in a single atomic operation, the value of `s`. The process that executes `V ( s )` never blocks.

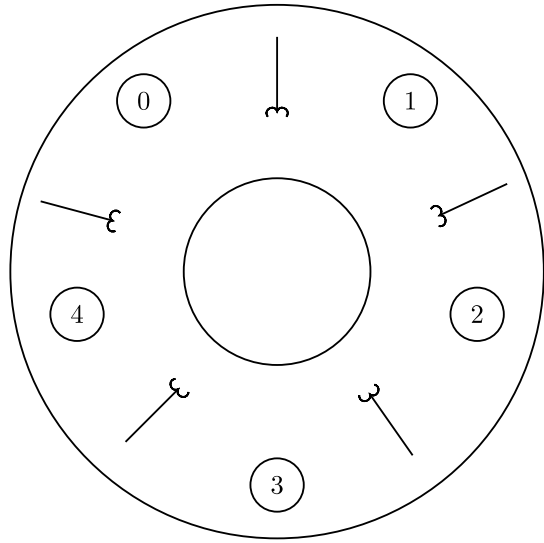
When the variable `s` can only assume values 0 and 1, we speak of a binary semaphore. When `s` can take non-negative values, we have a generic semaphore.

The “awakening” of the blocked `P` operations is normally handled by the process scheduler. Thus, the fairness of handling these operations, and the fairness of the

⁹ The names of these two operations come from Dutch, Dijkstra’s native language, and correspond to the initials of “verhogen” (V) meaning “increase” and “prolaag” (P). This second word is not actual Dutch but the abbreviation of “probeer te verlagen” (“attempt to reduce” or “attempt to decrease”). The abbreviation was invented by Dijkstra because also “decrement” begins with V in Dutch (“verlagen”) and he wanted one-word commands related to mnemonic correspondents.

¹⁰ In principle, semaphores could be implemented using active wait as well (at least on a multiprocessor machine). However, semaphores are usually implemented as mentioned above, hence the choice to put them in this paragraph.

Fig. 14.1 The dining philosophers problem



access of processes into critical sections, depends on the fairness assumptions of the scheduler itself. Normally, semaphore implementations ensure that the queue of blocked processes waiting for the completion of an operation P is managed in a First-In-First-Out (FIFO) manner: the first blocked processes are the first to be woken up. Thus, if there are enough operations V , fairness is guaranteed.

As an example of the use of semaphores, let us see the problem of the dining philosophers, a classic synchronisation problem due to Dijkstra. Five philosophers sit at a round table, as shown in Fig. 14.1. Each philosopher alternates between the activities of eating and thinking. Each philosopher can only use the forks placed at the sides of his plate, and each philosopher needs two forks to eat.¹¹ Since there are as many forks as there are philosophers, if each philosopher takes only one fork (e.g., the one on the right of the plate) no one can eat. This is an example of a deadlock situation. Each process (philosopher) must wait for the release of a resource (fork) by another process to proceed, but since the chain of release dependency is a cycle, no one can proceed. Hence, to avoid deadlock situations in this scenario, we need to avoid the circular wait (the first process waiting for the second which waits for the third and so on until the first again). We can achieve this, for example, by assuming that the first philosopher chooses the right fork first and then the left one, while all the others take the left fork first and then the right one. A solution to this synchronisation problem is therefore as follows. Let us assume that we have a `forks` array of 5 semaphores, one for each fork, which we all initialise to 1. We also denote with `philosopher[i]` the generic i -th philosopher (with $i \in [0, 4]$

¹¹ The traditional justification for using two forks is that the plate contains spaghetti, which are notoriously difficult to handle with only one fork by non-Italian philosophers.

by analogy with the indices of the array: fork number i is the one to the right of philosopher i). The selection of a fork is realised by the operation P while the release happens via the operation V.

```
sem[] forks[5];
for ( i = 0, i < 5, i += 1 ) { forks[ i ] = 1; }

process Philosopher( i ){
    sem first_fork, second_fork;
    if( i == 0 ){
        first_fork = forks[ 4 ];    // the one on the right
        second_fork = forks[ 0 ];    // the one on the left
    } else {
        first_fork = forks[ i ];    // the one on the left
        second_fork = forks[ i-1 ]; // the one on the right
    }
    while true {
        think;
        P( first_fork ); // acquire first
        P( second_fork ); // acquire second
        eat;
        V( first_fork ); // release first
        V( second_fork ); // release second
    }
}

Philosopher(0) || Philosopher(1) || Philosopher(2) || Philosopher(3) || Philosopher(4)
```

The last line of code indicates the concurrent execution of the processes corresponding to the five philosophers. Proving the correctness of the solution is trivial. Indeed, let us unfold one of the possible executions. We start from `Philosopher( 1 )`, who takes his first fork (the one on his left) `forks[1]`, followed by `Philosopher( 2 )`, who takes `forks[2]` (also on his left) and so on until `Philosopher( 4 )`, who takes `forks[4]` (on his left). Now it is the turn of `Philosopher( 0 )` to take his first fork. However, instead of completing the cycle (of left-first acquisitions, which would end in a deadlock), `Philosopher( 0 )` tries to acquire the fork on his right. This is `forks[4]`, which was already acquired by `Philosopher( 4 )`. The only thing that `Philosopher( 0 )` can do is to wait for that fork to become available, allowing the other philosophers to pick their second (right) fork (`Philosopher( 1 )` would pick `forks[0]`, `Philosopher( 2 )` `forks[1]` and so on, until `Philosopher( 4 )`). When `Philosopher( 4 )` and `Philosopher( 1 )` release their forks, `Philosopher( 0 )` will finally have the chance to pick his forks (right first, left second) and eat his well-deserved meal. We discuss more in depth on the operator `||` in Sect. 14.6.1.

Monitors

In the previous chapters, we saw how abstraction mechanisms play an important role also in the context of data structures. In particular, we have analysed in detail the basic idea of abstract data types first, and then of objects, trying to understand why it is important to be able to provide constructs that encapsulate both the abstract representation of a datum and the operations possible on it. From this point of view,

semaphores are deficient, since they are low-level constructs that do not offer sufficient structuring and abstraction mechanisms. In fact, a semaphore is essentially a variable shared by several processes, and it is the task of the programmer to ensure that this variable is used correctly, using P and V , with all the consequent risks on correctness (use of the wrong semaphore, omission of a P or V , use of operations other than P and V , etc.). Notice also that semaphores can work both to implement mutual exclusion and condition synchronisation. This is not completely satisfactory, since the two concepts are different and, to help developers understand a program, it would be good if this diversity also appeared at a linguistic level.

To overcome these shortcomings, the 1970s saw the introduction of the *monitors*, also initially proposed by Dijkstra, subsequently developed by Brinch Hansen and C.A.R. Hoare, and later used in many programming languages, including Java. A monitor is an abstract data type that encapsulates the representation of an abstract object with state and provides permitted operations to act on that object. More precisely, a monitor contains permanent variables, representing the state of the object, procedures, to act on these variables, and initialisation commands, used when the monitor is created.

Without referring to any specific language, we can represent a monitor declaration as a data type with the following structure

```
monitor MonitorName {  
    variables;  
    initialisation commands;  
    procedures;  
}
```

Permanent variables exist as long as the monitor exists (of course, procedures may have their own local variables that have a shorter lifespan). Among them there are certain variables known as condition variables that, as we shall see, allow condition synchronisation. Outside the monitor, only the procedure names (with their signatures) are visible, which are therefore the only operations that can be used by (external) processes to act on the permanent variables, similarly to what happens with the fields of objects defined with a `private` visibility. Permanent variables are initialised, by the relevant commands, when the monitor is created and before any procedures are used.

In a concurrent program using monitors, we can distinguish passive and active components. The former, introduced by declarations similar to the one seen above, are the monitors and contain the data structures shared between the various processes. The latter are the processes and these can interact using the procedures (and, indirectly, the variables) of the monitors.

Already from this first description, the advantage of monitors over semaphores is evident: the programmer using a monitor can develop a process in a relatively independent way, knowing only the external interface of the monitor, ignoring how its procedures are implemented and also, in part, how these procedures are used by other processes. This is the usual advantage of abstract data types and objects, but here it assumes further relevance: unlike what happens in sequential programming, the monitor is shared by various concurrent processes that are active at the same time with different timings and speeds. Thus, it is essential to be able to develop each process relatively independently, both from the point of view of understanding the

program and, above all, for its correctness. Of course, problems of mutual exclusion and synchronisation must be solved, but using monitors helps in these regards.

Indeed, mutual exclusion, using monitors, is implicitly guaranteed by the construct itself, without the programmer having to do anything: monitor procedures are executed in mutual exclusion *by definition*. More precisely, we define *active* a monitor procedure that is executed upon invocation by a process. The monitor specification guarantees that at any instant there cannot be two active instances of monitor procedures, whether they are instances of different procedures or the same procedure. The implementation of the monitor will, of course, have to guarantee compliance with this specification by using appropriate lower-level mechanisms (locks, semaphores or even interrupt inhibition).

On the other hand, even using monitors, the programmer must explicitly specify condition synchronisation, using the conditional variables of the monitor, which allow the execution of a process to be suspended while waiting for a certain condition to occur. However, even in this case, one can access variables only through a procedure defined within the monitor and only using specific operations. More in detail, a conditional variable can be declared inside the monitor with an appropriate syntax, for example,

```
cond varname
```

We can conceptually regard the value of such a variable as a queue of blocked processes. The most important monitor operations possible on such a variable are as follows:

```
empty( varname )
```

which returns *true* if the queue representing the value of `varname` is empty, *false* otherwise.

```
wait( varname )
```

A process that performs this operation is blocked and is placed at the end of the queue associated with the variable `varname`.¹² At the same time, the process releases the use of the monitor, which can then be used by (other) processes.

Finally, we have the operation that allows a process to wake up. This is

```
signal( varname )
```

which wakes up the process at the head of the queue `varname` if it is not empty, and has no effect otherwise. In this way, a process that was previously blocked with a `wait` can be re-executed by the `signal`, according to a FIFO queue management policy: the first process to be blocked is also the first to be woken up. The attentive reader will have noticed a problem at this point: since both the `wait` and the `signal` are operations that only procedures defined inside the monitor (but invocable from outside) can use, both the process executing the `signal` and the awakened process that had previously executed the `wait` are located “inside the monitor”. This, as we saw earlier, is not possible, since only one monitor process can be running at any time. In general, there are two techniques to solve this problem:

¹² The queue contains only process references.

Signal and Continue The process that sent the `signal` continues execution, while the other is only woken up “virtually”, i.e., its state is changed to indicate that it can be re-executed when it enters the monitor.

Signal and Wait In this case, the woken process executes immediately and gains access to the monitor. The process that sent the `signal` is instead blocked (and, in this case, we can put it at the head or tail of the queue).

14.5 Message Exchange

As we have said before, in the case of distributed architectures, be they multi-computers or networks, the communication model most commonly used is based on the exchange of messages. Here, we analyse this model in detail, focusing on the most important aspects that characterise it, i.e., the mechanisms for identifying partners (or naming mechanisms) and the sending and receiving mechanisms, both asynchronous and synchronous.

14.5.1 Naming Mechanisms

Two communicating partners, to be able to “speak” to each other, must have mechanisms in place to know to whom to send and from whom to receive messages. That is, we need mechanisms to designate, or name, the partners themselves. In this section, we look at three alternatives. In the following, we call the two communication partners *sender* and *receiver* and assume that these are processes (as usual, they could also be threads or something else).

Names

The first mechanism we examine is perhaps the most intuitive: processes have a name, consisting of a unique identifier and, to communicate with each other, they explicitly use this name as an argument of send and receive operations. This way of communicating was introduced in Hoare’s CSP language and then adopted in other cases, such as the PVM and MPI libraries. Thus, for example, we can have two processes A and B specified as follows

```
process A {
    ...
    B!v;
    ...
}
process B {
    ...
    A?y;
    ...
}
```

where process A sends the value `v` to process B, which receives it in the variable `y` (`!` and `?` in CSP notation indicate send and receive operations, respectively).

Ports

The second method of identifying the communication partner uses ports. An input port conceptually is a device that provides an entry point into the receiving process. Similarly, an output port provides an exit point for a sender process. The main use case for ports is that they allow, in a simple way, the same physical communication device to be shared between multiple processes. Take, for example, a transport layer communication protocol such as TCP. If a process *A* wants to establish a TCP connection with a process that is on a machine whose IP address is *foo* (we use symbolic names, of course), *A* must specify both that address and the port number *x* (between 0 and 65535) with which it wants to connect. This makes it possible to have many different processes on the machine at address *foo*, active at the same time (let us assume the machine allows multiprogramming), which also use the same physical communication device (e.g., the same Ethernet port). The “right” process, with which *A* wants to communicate, will be the one listening on port *x*.

Ada was one of the first languages to use ports through so-called *entry calls*. A task (Ada’s equivalent of a process) can in fact specify an input port as follows:

```
task TaskType is
  entry inputPort( data: in integer );
  entry outputPort( data: out integer );
end TaskType;
```

This specifies that a task of type *TaskType* has two ports named *inputPort* and *outputPort*. If the task body is then defined as follows

```
task body TaskType is
  ...
  accept inputPort( data: in integer ) do ... end inputPort
  ...
  accept outputPort( data: out integer ) do ... end outputPort
  ...
end TaskType;
```

then, the command *accept* indicates that a process *Pr*, of type *TaskType* can receive from port *inputPort* a message containing a variable *data* of type integer. After receiving this value, the part of the program following the command *do* executes. This is similar for *outputPort*, with the difference that, in this case, the parameter *out* specifies a mode of passing from the called (i.e., message receiver) to the caller (sender).

To send a message to the process *Pr*, the sender process will simply execute a command of the form

```
Pr.inputPort( D );
```

where *D* is a variable containing the value we wish to pass on. Note that each specific process has its own ports, identified by the process type declaration and process name. Thus, messages sent as seen above only reach one process.

Channels

The last method we see for establishing communication uses *channels*. These first appeared at the programming-language level in Occam, and have subsequently been

used extensively in various theoretical models (e.g., the π -calculus) and languages. A channel, conceptually, is an abstraction of any communication structure that allows two partners to be connected. In this sense, even a shared variable or a monitor may be considered as an implementation of a channel. Normally, however, the term channel is reserved for those models that are abstractions of a real communication network, in which we have physical channels connecting separate devices. In what follows, we do not make any assumptions about how channels are implemented but we limit ourselves to an abstract use of this term.

Trying to be a little more precise but, as usual, without referring to any particular language, we can see the code

```
channel chanName ( Type )
```

as the declaration of a channel where the identifier `chanName` denotes the name of the channel and the `Type` of messages sent over that channel. Of course, the limitation to one field is made for the sake of simplicity: we may have fields of different types, identified by appropriate names.

The sender and receiver must both know the name of a channel in order to communicate. Referring to the previous statement, communication is done by sending data to the channel `chanName`, with the operation

```
send chanName ( data )
```

performed by the sender, and receiving data from the channel, with the

```
receive chanName ( var )
```

executed by the recipient. The variables `data` and `var` of type `Type`: the first contains the values to be transmitted, while the second contains the values received after the communication has taken place.

In practice, there are different channel models, corresponding to the different communication mechanisms that exist. In particular, we can distinguish *one-way channels*, which allow data to flow in only one direction, from *two-way channels*, which allow data to flow in both directions.

Note also that, terminology-wise, a one-way channel connecting a sender to a receiver is called a *link*, while sometimes a (one-way) channel connecting several senders to a single receiver is called an *input port*. Finally, we call a channel *mailbox* when we have many-to-many communication, where multiple senders can send to multiple recipients. Below, we assume that channels are of this type unless otherwise specified.

From the point of view of semantics, we can see a channel as a queue of messages that have been sent and have not yet been received. The length of such a queue depends on whether the communication is asynchronous or synchronous, as we shall see in the next two sections.

14.5.2 Asynchronous Communication

In this and the next section, we analyse the two main ways of communicating by exchanging messages. We do this by referring to a communication structure that uses

channels, however, the following considerations essentially also apply when using explicit process names or ports.

In asynchronous communication, the sender, after sending a message, continues with the computation without waiting for the receiver to have performed the receiving operation. In other words, with asynchronous communication the operation

```
send chanName( data )
```

is non-blocking: the effect of this operation is to add a message containing the value of the variable `data` to the list of messages associated with the channel `chanName`, after which the process that performed the `send` operation can continue its computation. We assume, here, that the operation that accesses the channel is atomic and also that the operation that sends the message is reliable (i.e., sent messages are actually received, without errors).¹³ We also assume that the channel queue is FIFO, so that the first message sent is the first received. Of course, the accuracy of these assumptions depends on the underlying structure implementing the channels. Note also that, at least in principle, with asynchronous communications the queue associated with the channel has an unbounded length, since an arbitrarily large number of messages may be sent before any reception.

Reception, as mentioned before, works using an operation

```
receive chanName( var )
```

The effect of this operation is as follows. If the queue associated with the channel `chanName` is not empty, the `receive` operation fetches and removes the first message from the queue. The data for that message is assigned¹⁴ to the variable `var` (also in this case, we assume that the access to the queue is achieved by an atomic operation). If, however, the message queue is empty, then the `receive` operation is blocked and the process that performed it is blocked as well, until there is at least one message in the queue. When this event occurs, the blocked process is woken up—following the modalities already described earlier, when we looked at block-based synchronisation. Since the `receive` operation is always blocking, in some cases, we want to give the process the possibility to check if the channel is empty or not, to avoid performing (and blocking on) a `receive` on an empty channel. Similarly, we can let processes define a timeout for the `receive` operations, which would interrupt the wait on an empty channel after a fixed amount of time from its invocation. Some languages provide specific constructs to deal with these cases. Furthermore, in some cases, the operation of receiving a message is not done explicitly using a communication primitive but implicitly. This case will be considered in Sect. 14.5.4, when we discuss Remote Procedure Calls.

As an example of the use of asynchronous communication, we see a simple interaction between two clients and a server. Each client sends a request to the server

¹³ The assumption on the atomicity of the operation on the channel implies that for a short period (depending on how the channel is implemented), the sender process must wait before continuing. We ignore this waiting time here.

¹⁴ Clearly, we use this term informally here, as we may have imperative languages, but also functional or logical one.

specifying the name of the client (1 or 2) and communicating data to be processed (of type `char`). The server responds to the client by sending the processed data.

```
channel request( int client, char data );
channel response1( char res );
channel response2( char res );

process Client1{
  char values;
  char results;
  ... // Definition of values
  send request( 1, values );
  receive response1( results );
  ... // Usage of results
}
process Client2{
  char values;
  char results;
  ... // Definition of values
  send request( 2, values );
  receive response2( results );
  ... // Usage of results
}
process Server{
  int client;
  char values;
  char results;
  ... // Initialisation
  while true {
    receive request( client, values );
    if client = 1 then {
      ... // Processing of values
      send response1( results );
    }
    if client = 2 then {
      ... // Processing of values
      send response2( results );
    }
  }
}
```

There are a few things to note in the example above. First, both clients use the channel `request` to send requests to the server. The server, on the other hand, sends replies using one channel per client. Furthermore, the channel names are static and global to the whole program, so the correctness of communication depends on their correct use. The process `Client1`, for example, could use the channel `response2`, which should be reserved for answers for the second client, without there being any control over this. To avoid possible errors, and also for confidentiality, each client could include in their message to the server the name of its local channel (not visible to other clients) to indicate where the server shall send the reply. This, however, would require dynamic name management, which is not generally simple. Finally, the example is given for two clients for the sake of simplicity, but one can simply extend it to n clients, statically known, using an array of n channels. On the other hand, it is more complex to extend the example to manage several clients not known beforehand (again, we would need dynamic name management to allow each client to tell the server the name of its channel).

14.5.3 Synchronous Communication

In synchronous communication (with the exchange of messages), the `receive` operation is the same as in the asynchronous case. However, the `send` operation changes to be blocking as well, i.e., the sender, after sending a message, blocks until the receiver receives it. Therefore, at least from a conceptual point of view, in the synchronous case, we can see `send` and `receive` as two operations that allow the processes to synchronise when the communication is realised. From a real-world perspective, the operations take place at different times and the sender, after having carried out the `send`, waits for a signal confirming the reception of the message. Synchronous communication first appeared in CSP.

The main advantage of synchronous over asynchronous communication is that we can limit the maximum length of the message queue associated with a channel. In fact, any process that sends a message cannot send another until the previous one has been received and removed from the queue. Thus, the queue will contain at most one message per sending process. Another advantage of synchronous communication is error handling. If the reception of a message causes an error, if the sender is stuck waiting for the communication to complete, it is easier to handle the anomalous situation than if the sender has continued with its own computation.

Of course, synchronous communication also has disadvantages. First, the degree of concurrency among processes is reduced due to the blocking `send`: since `send` and `receive` operations execute at different times, when a communication takes place, there is always at least one blocked process. This is not the case for asynchronous communication, when the `send` operation executes before the `receive`. This limitation can be relevant in real-world applications. Suppose that, following a typical concurrent interaction pattern, we have a *producer* process that sends data and a *consumer* process that receives it. If the two processes have different processing times, there is no guarantee on the order in which `sends` and `receives` execute. In any case, with each synchronous communication we have a delay due to one of the processes stopping. If, on the other hand, we used asynchronous communications, we would reduce the total delay, since the queue associated with the channel would act as a buffer and mitigate the discrepancies of the different speeds: when the producer is faster than the consumer, the messages queue in the channel; in the opposite case, the consumer can pick as many messages (when present) as it can process. We can obtain a similar behaviour with synchronous communication, by interposing a buffer process between producer and consumer, however, even in this case, the efficiency is lower than in the asynchronous case.

In general, efficiency aside, we can simulate asynchronous communication through synchronous communication using a process that realises the asynchronous channel and responds immediately to both senders and receivers of the channel. It is also possible to do the opposite, since we can always pair a non-blocking `send` with a `receive` that waits for an acknowledgement of reception from the receiving process, thus realising synchronous communication.

A second disadvantage of synchronous communication is the greater difficulty of usage for the programmer. Or rather, the programmer must pay greater attention

because, with synchronous communication, it is easier to generate *deadlocks* and other errors. As an example, let us consider two processes that communicate by both executing `sends` and `receives` (this happens in almost all peer-to-peer applications). If both processes perform a `send`, we have a deadlock where the computation cannot proceed any further, as each process is stuck waiting for a `receive` from the other. Such an error would not occur with asynchronous communication since, in this case, the `send` operation is non-blocking (of course, other deadlock situations can also occur with asynchronous communication).

14.5.4 Remote Procedure Call and Rendez-Vous

We now look at two communication mechanisms that are widely used in practice, especially in client-server interactions: the Remote Procedure Call (or RPC), used in many languages (including Java), and the rendez-vous, introduced and extensively used in Ada. To a first approximation, we can see both of these mechanisms as methods of synchronous communication via message exchange. Here, unlike the synchronous `send` seen in the previous paragraph, the sender, after sending a message, waits for a reply message from the receiver (and not just a confirmation that the message has been received) before proceeding with the computation. More precisely, and as the names of these operations suggest, both RPCs and rendez-vous invoke a procedure that is performed remotely, typically by a process that runs on another machine, even geographically distant. The difference between RPC and rendez-vous lies in the way the remote call is handled.

In the case of RPCs, the recipient generally does not have an active process waiting for that specific call. Contrarily, when the recipient receives the message, it creates a new process to handle it. This process is responsible for passing the parameters, executing the body of the called procedure, and transmitting the results to the caller. Once this is done, the process terminates its execution and any new call would trigger the creation of a different, dedicated handling process. Of course, the underlying communication infrastructure must handle the reception and the triggering of the recipient upon the reception of new messages, usually done via mechanisms such as ports and channels. In terms of message exchange, we can say that RPCs perform only explicit `send` operations, called `calls`, but no explicit `receive` operations.

With rendez-vous, on the other hand, the calling process performs a rendez-vous with an already-existing process, active in the receiver. When the call arrives, the process executes the requested procedure (after passing the parameters, as usual), it transmits the results to the caller, and then continues its activity, e.g., to handle new requests. So, in this case, in addition to the sender-side `call`, there is also an explicit `receive` operation at the receiver, such as the `accept` operation in Ada, already seen in Sect. 14.5.1. Similar to the case of the `receive`, the `accept` is suspended if there are no requests (`call`) to be handled.

In the following, we see some further details on RPCs, and we refer the interested reader to specialised literature for what regards rendez-vous.

To realise a remote procedure call, the process holding the procedure's code must publish at least the name of the procedure itself to the external environment, so that it is visible and usable for the call. This is normally done in a programming context that uses modules and, in the following, we make the assumption of having these constructs as well. A module is therefore a component (of a program) that contains both processes and exportable procedures. These are explicitly defined in the module itself and are distinct from local procedures that can only be used within the module. Furthermore, the processes of a module are also divided between purely local processes, called background processes, and processes that are created to serve outside procedure invocations. Different modules may reside on different machines and may communicate with each other by invoking remote procedures. A module, therefore, has the following structure:

```

module modName{
  op proc1( type of formal parameters ) returns ( type of result )
    // headers
  ...
  body {
    local variables;
    initialisation code;
    proc proc1( formal parameters ) returns ( result ){
      procedure body
    } // implementation
    ...
    local procedures;
    local processes;
  }
}

```

As can be seen, the initial part lists the names of the exportable procedures with the names and types of the formal parameters and returned results. Then, in the body, with the keyword `proc`, we define the implementation of each procedure (note that the type of the formal parameters and results is not repeated here, only their name).

Referring to the code above, another module may call the procedure `proc1` of the module `modName` using the command `call` in which these two names and the current parameters are specified:

```

call modName.proc1( actual parameters )

```

As mentioned earlier, such a call causes the creation of a new process which, after receiving the current parameters, executes the body of the procedure `proc1`. When this finishes, the process sends the result to the caller and terminates. The caller, which was stuck after the execution of the `call`, once it has received the results, can continue its execution.

Note that, if the various active processes in the same module—both running in the background and the ones executing remote calls—execute concurrently, we must use appropriate synchronisation and mutual exclusion mechanisms, since these are in no way provided by the RPC. Alternatively, we may assume that, in each module, there is at most one active process at any time, similar to what happens with a monitor.

As a final point, let us notice that in cases where RPCs are not directly supported by the language, we can obtain them by adding so-called *stubs* to an existing language, to make the remote procedure call as similar as possible to a “normal” procedure call,

hiding from the programmer the communication details needed to handle the RPC. Stubs are usually generated automatically by a *stub compiler*. When the program makes a remote call, it actually calls a stub, which takes all the parameters necessary for the exact identification of the operation (name of the called procedure, actual parameters, etc.), it builds an appropriate message containing them, and it sends the message to another stub (sometimes called a skeleton) located at the recipient. The second stub receives the message, extracts the parameters, and then calls the local procedure resulting in the creation of a new process. When the execution is complete, in a dual manner with respect to the first interaction, the second stub creates a message with the results and it transmits it to the first stub, which finally extracts the results and passes them to the process that invoked the RPC.

14.6 Non-determinism and Parallel Composition

In this section, we look at two fundamental aspects of the specification of concurrent computation. We begin with non-determinism.

The constructs for controlling computation that we have seen in previous chapters, and in particular in Chap. 7, all have one common characteristic: they are deterministic. This means that at every moment of the computation, for a given current state, we unequivocally know what is the next instruction to be executed and there is never the possibility to choose between two or more equally-possible alternatives. Take, for example, the conditional choice command `if then else`: the evaluated condition is either true or false and, depending on the value, we execute one of the two alternatives. It is never the case that both branches `then` and `else` are equally possible as a continuation of the computation. In other words, in a deterministic computation, the initial state, in the absence of further external input, uniquely determines the final state.

Beyond philosophical considerations on the ultimate nature of our computing devices, the deterministic view allows us to simplify the model of computation. However, there are situations, especially in concurrent environments, in which such a vision is limiting. Consider, for example, a server that, using message-exchange communication, receives both read and write requests from two distinct clients (e.g., to access a shared file). Requests may arrive in any order and at unpredictable times. Furthermore, for confidentiality reasons, the channels used by the two clients to send the requests must be different. Following the scheme of the program seen in Sect. 14.5.2 we could implement the server as follows:

```

channel reading ( int data );
channel writing ( int data );
process ReadingClient{
  int results;
  ....
  send read( results );
  .... // usage of the results
}
process WritingClient{
  int values;
  .... // definition of values
  send write( values );
  ....
}
process Server{
  int results;
  int values;
  while true {
    receive read( results );
    ... // serve reading request;
    receive write( values );
    ... //serve writing request;
  }
}

```

However, this solution does not work well. In fact, we imposed an order for the reception of read and write operations. For example, if a client sends a `write` request but no `read` request was sent prior to that, the server will keep that request waiting until it can perform the receive operation on the `read` channel. The wait may also become a deadlock if the send request on the `read` never arrives (e.g., because the corresponding process has aborted its execution).

To solve this kind of problem the most natural and elegant solution is to use a construct that allows introducing non-determinism in the choice among several alternatives. One of the first proposals in this sense, dating back to the 1970s, by Dijkstra, is the guarded command, which has the following structure:

```

condition -> command
[]
condition -> command
[]
...
condition -> command

```

The meaning of this command is as follows. The condition (also called guard) to the left of the arrow is a Boolean expression that is evaluated. If the condition is true, then the corresponding branch can be chosen and the command to the right of the arrow can be executed. If more than one condition is true, and thus more than one branch can be executed, then one of them is chosen in an unspecified way, i.e., in a non-deterministic way. Obviously, in the presence of such non-determinism, given an initial state, the computation can reach several final states.

In a more recent version of this command, the guard not only contains a Boolean condition, but also a communication command that can also be blocked. Then, a guard can fail if the Boolean condition has a false value, but it can succeed if the Boolean condition has a true value and the communication command can be executed without suspension—and it can block if the communication command blocks. If all

the guards are blocked, then the whole command is blocked, whereas if there is at least one successful guard, similarly to what was seen before, a branch is chosen, non-deterministically, from among all those whose guards that are successful. If all guards fail, the command has no effect.

Using such a command we can program our server as follows

```
... // same as before
process Server {
  int v;
  int r;
  while true {
    receive read( results ) -> ... // serve reading request
    []
    receive write( values ) -> ... // serve writing request
  }
}
```

In this way, we eliminate the problems mentioned above because, as soon as one request is made, it can be served without having to wait for the other.

The discourse on non-determinism and its expressive power would take us far beyond the scope of this chapter. Here, we merely observe that, at the implementation level, appropriate, non-trivial techniques (e.g., pseudo-causal number generators) are needed to handle it correctly. We also observe that when we use a command with a guard in the context of an infinite loop—as in the server example—a fairness problem also arises: if a guard is always verified, we would like the corresponding branch to be chosen sooner or later. Again, various notions of fairness and various solutions have been proposed.

Let us conclude this paragraph with another example that shows, even more than the previous one, the usefulness of non-determinism. Suppose we have two processes that produce two streams of data that we wish to merge into a single stream. The process that performs this “merging” is a particular type of filter, called a merge (in general, a filter is a process that receives data from one or more input channels and sends it to one or more output channels). The streams we consider are sequences, even infinite ones, of data, so it is not possible to receive first all the data of one channel and then all those of the other. Moreover, as it is often the case in practice, the individual data on each channel are sent at irregular intervals, without any synchronisation or without the possibility of making assumptions on when they will arrive. We can use the command with guard to program the merge process as follows:

```
channel input1 ( int data );
channel input2 ( int data );
channel output ( int data );

process Server{
  int data;
  while true {
    receive input1( data ) -> send output( data )
    []
    receive input2( data ) -> send output( data )
  }
}
```

Note that, in this way, as soon as one of the two input channels produces a piece of data, this is sent out without having to wait for the other channel. In the edge case in which no more data arrives at one channel, the process continues to transmit data from the other channel. This would not have been possible by imposing a static ordering between the receptions on the two input channels. Finally, of course, the merge process can be arbitrarily complicated by inserting elaborations on received data or by increasing the number of channels.

14.6.1 Parallel Composition

The second important construct for the definition of concurrent computations is the parallel composition operator, often denoted by $\parallel$.¹⁵

Intuitively, the $\parallel$ operator specifies that the processes constituting its operands should be executed in parallel, i.e., concurrently. However, from a semantic point of view, there are various possible interpretations for this. The interpretation adopted in the case of parallel programming is what is usually referred to in theoretical works as *maximum parallelism*: all processes that appear in the parallel composition and which may proceed in the computation, do so. Of course, this interpretation assumes an underlying multi-processor architecture. Indeed, in principle, the case of dynamic process creation requires an architectural model with an unbound number of processors.

Therefore, without making any assumptions about the underlying architecture, we can define a different semantic model of the $\parallel$ operator, called *interleaving*. According to this model, at each instant only one of the processes can proceed in the computation, possibly alternating its actions with those of other processes that appear in the parallel composition (to interleave means precisely “to alternate different levels”). Thus, if the process P can do (a computation described by) a sequence of actions s_1 and the process Q can do the sequence s_2 , the process $P \parallel Q$ can do any sequence obtainable by alternating arbitrarily the actions s_1 and s_2 , provided the order given by s_1 and s_2 themselves is respected. As a concrete example, if P can do the sequence of actions $abac$,¹⁶ while Q can only do d , then $P \parallel Q$ can do the sequences $dabac$, $adbac$, $abdac$, $abadc$, and $abacd$. Note that interleaving introduces a further level of non-determinism into the computation, since even if P and Q are deterministic processes (each having only one possible computation for a given input), the parallel composition $P \parallel Q$ can still have more than one possible computation depending on how one chooses to alternate the actions of the two processes (consider $R \parallel Q$, where Q is the one from before and the process R can only do the sequence ab).

¹⁵ This operator is binary but, being associative and commutative, we can omit the parentheses in the case of multiple occurrences of the same one.

¹⁶ Here, $abac$ means that the process first executes a , followed by b , a again, and finally c .

14.7 Concurrency in Java

In this section, we attempt to concretise some concepts seen in this chapter by examining the case of a real language. Of course, we will only present some main aspects. For an in-depth understanding of concurrency in Java, there are specific references in the bibliography.

14.7.1 Creation of Threads

In Java, a sequential program contains a single thread used to execute the program's main method. If, however, we create several threads that execute within the same program, then we have a form of concurrent programming (multithreaded).

Technically, a thread is an object of class `Thread` provided by the `java.lang` package. To create a thread we can simply create an object of this class, for example by declaring:

```
Thread foo = new Thread();
```

Once we created a `Thread` object, and possibly defined some initial parameters (e.g., priority), we can execute the thread invoking the method `start`, defined in the class `Thread`, as in the following code:

```
foo.start();
```

More in detail, the method `start` generates a new thread using the data provided in the `foo` object and it invokes the method `run` (also defined in the `Thread` class) for the `foo` object and then terminates. At this point, the new thread is made active by the Java Virtual Machine, which executes `run`. Note that the method `run` is directly accessible to the programmer, but invoking it directly would make the current thread execute its code, rather than creating a new one (as the `start` does). Furthermore, each object of type `Thread` can only invoke the `start` method once.

Leaving things as they are, however, does not produce any effect, because the method `run` defined in the `Thread` class does nothing. To get some useful behaviour, we have to define the body of the method `run`. We can do this in two ways.

The first is to extend the class `Thread` via a subclass in which we redefine the method `run`. For example, if we have

```
class MyThread extends Thread {
    public void run() {
        System.out.println( "I am thread foo" );
    }
}

Thread foo = new MyThread();
foo.start();
```

the `Thread foo` is executed and its `run` method prints the string `"I am ..."`.

The other way to define a useful behaviour of the method `run` is by using the interface `Runnable`. This interface, also provided by the `java.lang` package, represents in the abstract a task, a job to be performed by the method `run`, which is declared in the interface as follows:

```
public void run();
```

This work must be executed by a specific thread which, continuing the metaphor, can be seen as the worker performing it. To do this, we must first define an object of an arbitrary class, say `C`, that implements the `Runnable` interface and then provide specific code for the `run` method. Next, we need to create an object of class `C` so that we have a specific instance of our work. Then, we need to pass the object thus created to a specific object of class `Thread` via the constructor of this class. Finally, the work can begin using the `start` method, as seen above.

The example from before, realised using the `Runnable` interface, becomes the following:

```
class C implements Runnable{
    public void run() {
        System.out.println( "I am thread foo" );
    }
}
Runnable c = new C();
Thread foo = new Thread( c );
foo.start();
```

Note that `c` has static type `Runnable` and dynamic type `C` and that this object is passed to the constructor of the class `Thread` to allow the method `run` to be executed. This method, as before, writes the string when the thread is activated by the `start` method.

The advantage of this alternative method is the possibility of having several interfaces implemented by a single class, allowing the definition of a class that has executable objects (as an implementation of `Runnable`) and that, at the same time, implements other interfaces that define other characteristics of the objects. The first approach cannot implement such behaviour, since, as we have seen, Java supports single inheritance (cf. Sect. 10.2.5 and following). Moreover, the class `Thread`, which also implements `Runnable`, often contains much more than what is needed to execute a thread.

14.7.2 Scheduling and Termination of Threads

Normally, multithreaded systems have more threads than processors, so it is necessary to define *scheduling* policies that allow us to appropriately divide CPU time between the various threads running. These policies, in detail, depend on the particular implementation of the Java Virtual Machine and the platform on which we are running our Java applications. In general, we can say that each thread has a priority that is used by the system at runtime to determine which thread to execute next. However, some mechanisms limit the amount of CPU time that can be used by a single thread to avoid the so-called *starvation* of lower-priority threads, i.e., to prevent the

latter from never being executed in the presence of higher-priority executable threads. An executing thread, therefore, continues to use the CPU until it either performs a blocking operation that, as we have already seen, causes the CPU to release, or until it receives an interrupt from the system, an operation normally called *preemption*. This interruption occurs either because a thread with a higher priority has become ready to be executed or because the system scheduler has decided that the executing thread has exceeded its CPU usage quota. Threads may also voluntarily release the CPU using specific methods such as `sleep`.

The priority of a thread is initially the same as the thread that created it and can be changed using the command `setPriority` to set a value between the constants `MIN_PRIORITY` and `MAX_PRIORITY`. The use of this command must be done bearing in mind that the actual execution of the various threads depends not only on the priority but also on the policy used by the scheduler. Therefore, while it certainly makes sense to assign a higher priority to a thread that we wish to execute before another one (e.g., the one corresponding to a function that interrupts the normal execution of our application), we must not think that the values expressed by priorities are absolute.

A thread terminates and thus permanently releases the computing resources it is using when it terminates the execution of the method `run`. However, there are other termination modalities.

First, using the `interrupt` method, an interrupt signal can be sent to a thread which, even if it does not directly cause the termination of an active thread, can be appropriately handled in the program of the receiving thread to cause termination. For example, if we have two threads `t1` and `t2`, we can have in `t1` the code

```
t2.interrupt()
```

while in the thread `t2` we can have

```
...
while ( ! interrupted() ){
    ...
}
```

This way, when `t1` invokes (or, more commonly said, “throws”) the interrupt, the `while` loop is interrupted and the thread `t2` can terminate (if there is nothing else to do after the `while`). Note that an interrupt causes the waiting period of a blocked thread to be interrupted. Furthermore, if the thread is executing a `sleep` or `wait` method, when it receives an interrupt request a `InterruptedException` is thrown, which must be handled appropriately.

A second aspect of thread termination concerns the fact that, normally, in a multithreaded application we have a `main` thread and other threads that are launched from this and executed concurrently. If the `main` thread terminates, what happens to the other threads depends on their nature. Here, we distinguish between user threads and daemon threads. Daemon threads are threads defined as *daemons* through the invocation of the method `setDaemon( true )` and all those threads created by daemons. User threads are all the threads not specifically set as daemons. The latter continue their execution even if the `main` thread has terminated, while daemon threads terminate immediately when there are no more active user threads.

14.7.3 Synchronisation and Communication Between Threads

As we have seen in previous sections, in presence of concurrent threads, we need appropriate synchronisation and inter-thread communication mechanisms to prevent incorrect access to shared data. The solution adopted in Java is similar to what we have seen in the case of monitors, although there are some peculiarities due to the interplay with the object-oriented paradigm.

Usually, the shared data for which we want to guarantee mutual exclusion in Java are objects (but there are also static fields). We have a *lock* associated with every object that, to guarantee synchronization, must be acquired before the object is used and released afterwards. However, as in the case of monitors, the Java programmer does not have to make these operations explicit, and they must simply use *synchronized* methods (or commands). Such methods are declared in the class where we want to guarantee mutual exclusion. When we invoke a *synchronized* method on an object in a specific thread, before accessing the object, the method performs an (implicit) lock acquisition operation on the object. When the method terminates (either normally, or by throwing an exception) the lock is released. If the lock on an object has already been acquired, invocations from other threads on any synchronized method of the same object are locked, waiting for the lock to be released. Methods that are *not* synchronized can proceed normally. Note that the lock on an object is possessed by a thread and not by a single method: so, if a synchronized method that possesses a lock on an object invokes another synchronized method on the same object (in the same thread) the computation proceeds without waits (indeed, the thread already has the lock on that object).

As an example of using synchronized methods, let us see a program in which we want to implement a counter that multiple threads can access, concurrently, to read the value and increment or decrement it.

```
class Counter {
    private int c;
    public Counter ( int startingValue ) {
        c = startingValue;
    }
    public synchronized int read() {
        return c;
    }
    public synchronized void increment() {
        c++;
    }
    public synchronized void decrement() {
        c--;
    }
}
```

As seen above, the constructor of a class does not have to be declared synchronized, since it is invoked by only one thread for each specific object. The other methods declared synchronized, guarantee mutual exclusion in accessing the counter. At this point, continuing in the example, we can create the threads that use the counter:

```

class Incr extends Thread {
    int value;
    Counter counter;
    public Incr( int v, Counter c ){
        value = v;
        counter = c;
    }
    public void run(){
        for ( int i = 0; i < value; i++ ) {
            counter.increment();
        }
    }
}
class Decr extends Thread {
    int value;
    Counter counter;
    public Decr( int v, Counter c ){
        value = v;
        counter = c;
    }
    public void run(){
        for ( int i = 0; i < value; i++ ) {
            counter.decrement();
        }
    }
}
class Main {
    static Counter counter = new Counter( 0 );
    public static void main( int incrNumber, int decrNumber ){
        Thread inc_thread = new Incr( incrNumber, counter );
        Thread decr_thread = new Decr( decrNumber, counter );
        inc_thread.start();
        decr_thread.start();
    }
}

```

As can be seen in the code above, the methods `run` of the two classes extending `Thread` define the behaviour for the two types of threads, namely incrementing and decrementing the counter. The value of the increment is passed as a parameter to the main method and, thus, to the constructor of the class `Incr` (the case of decrementing is similar), which creates a new object of that class that is also a new thread. The constructor of the `Incr` object also passes a reference to an object of type `Counter`, which is the one that contains the counter that the two threads perform their increment and decrement operations on. Mutual exclusion is guaranteed by the fact that the methods declared in `Counter` are synchronized. The method `start`, as we have already seen, starts the execution of the threads.

Keep in mind that, in the previous example (and in general), although mutual exclusion is guaranteed, we cannot make any assumption on the order in which the methods are invoked in the different threads running in parallel. As mentioned earlier, this depends on the thread scheduler; so if we want the threads to respect a specific order we must explicitly schedule them in that way.

We can also declare static methods as synchronized to ensure synchronization in accessing static data shared by multiple threads. In addition, as mentioned earlier, portions of code, more limited than an entire method, can also be declared as synchronized. In this case, the syntax is as follows.

```
synchronized( expression ){  
    commands  
}
```

The meaning of that `synchronized` command is analogous to that of `synchronized` methods, with two important differences. First, the object on which we request lock acquisition is not the current one (i.e., `this`) but it is the object whose reference is the result of the evaluation of `expression` (which, of course, must be of the right type). In this way, we can then express synchronization on multiple objects through a single method, thus allowing us to quickly specify sophisticated interactions. The second difference is in the fact that, here, the portion of code that is executed in the critical section (between the acquisition and release of the lock) is limited by the scope of the construct. This has positive performance consequences, since having shorter critical regions makes it possible to limit the waiting time for threads that want to access the same data.

The `synchronized` methods allow mutual exclusion but not conditional synchronization. To accomplish this, thus allowing threads to communicate directly with each other, Java provides the specific methods `wait`, `notify`, and `notifyAll`, all defined in the class `Object`. The `wait` method, similar to the `monitors` command of the same name, blocks the execution of a thread until a certain condition occurs. The methods `notify` and `notifyAll`, on the other hand, are similar to the `signal` command and notify one or all the blocked threads that a condition has occurred, waking them up.

In more detail, these three methods are invoked through a specific object and must be executed in `synchronized` portions of the code, which means we already acquired the lock on that object.

The `wait` method releases the lock on the object and suspends execution of the thread, which is then placed in the wait queue associated with the object. The queue is usually handled with FIFO policy, but there can be exceptions since, as seen, different priorities can be used. In Java, there are no explicit conditional variables as in monitors, however, we can think of there being an implicit conditional variable for each object.

The method `notify` awakens a thread from among those in the wait queue of the object on which the method itself is invoked. The awakened thread cannot be chosen and it is the first one in the queue. Given the queue management, which does not always guarantee the FIFO policy, in practice, one must be very careful in using this method to wake up the desired thread. The method `notifyAll`, on the other hand, wakes up all threads waiting on the object, so it is easier to use. In the case of both `notify` and `notifyAll`, the thread invoking the method retains the lock on the object, so the awakened thread(s) can only be executed later when they manage to acquire the lock on the object. Using the terminology seen for monitors, these methods have signalling and continuous semantics.

Note on Distributed Programming in Java

The constructs we have seen so far enable multithreaded concurrent programming. Java does not contain specific primitives for distributed concurrent programming; however, some packages support it. In particular, the package `java.net` allows communication with message exchange either using datagrams and the UDP protocol or using streams and TCP. This package provides the classes `Socket` and `ServerSocket` that allow many client/server type applications to be programmed using sockets (basically terminals of a connection). With the first of the two classes, we can create a socket by providing a hostname (or IP address), a port number for that host, and a local port number with which to associate the socket itself. This is then used, typically on the client side, to communicate data. The `ServerSocket` class, on the other hand, allows a particular socket to be created to sit on a port, on the server side, waiting for requests for connections.

Java also contains two packages, `java.rmi` and `java.rmi.server` that support remote procedure calling, here called RMI (Remote Method Invocation). The operation is similar to the one we described in Sect. 14.5.4, with the command `rmic` being the analogue of the stub compiler there described. Please refer to the Java manuals for further details on these packages.

14.8 Summary

In this chapter, we have looked at the main issues in concurrent programming, an important topic that would require far more space than we have been able to devote to it to be treated comprehensively. We examined the following aspects.

- The notion of threads, process, and the three main modes of concurrent programming: parallel, multithreaded, and distributed.
- The synchronization mechanisms used in shared memory models and in particular active waiting, semaphores, and monitors.
- The communication mechanisms used in message exchange models and in particular asynchronous communication, asynchronous communication, remote procedure call, and rendezvous.
- The non-deterministic computation models and guarded commands.
- The main constructs for multithreaded programming in Java, namely thread creation, termination and scheduling primitives, and synchronized methods.

14.9 Bibliographical Notes

The bibliography on concurrent programming is extensive and ranges from purely theoretical aspects to practical manuals. Particularly important in the area of the the-

ory are the, so-called, *process algebras*; formalisms that make it possible to describe concurrent processes and that are endowed with algebraic laws that allow properties of computations to be formally verified. Among the most important approaches in this area are Hoare's CSP [1], Milner's CCS, and, more recently, the π -calculus [2]. Also very important were the many contributions of Edsger W. Dijkstra to the theory of concurrency: in [3] semaphores were introduced, in [4] the idea of monitors (under the name of "secretary"), and in [5] guarded commands.

Several educational texts deal comprehensively with concurrent programming issues. These include [6], which gives a broad overview of the main techniques used, with precise references to various current languages. For concurrent programming in Java, in addition to the classic handbook [7] one can consult [8].

14.10 Exercises

1. Consider the following code snippet:

```
int x = 4;
int y = 6;
<x = x+y> || <y = y-x>
```

where $\parallel$ denotes parallel composition and assumes that all commands enclosed in the parenthesis $\langle \rangle$ execute atomically. What are the possible results? Does removing the angle brackets change the results?

2. Referring to the notation of the previous exercise, consider the following code fragment:

```
int x = 2;
int y = 8;
while (x != y) {x = x+1;} || while (x != y) {y = y-1;}
```

Does the computation end? Justify the answer.

3. Suppose we have an instruction $\text{swap}(x, y)$ that atomically swaps the contents of the two variables x and y (using an internal register to perform the swap). Using such an instruction realize the procedures for acquiring and releasing the lock for access to a critical section (see Sect. 14.4.1).
4. Simulate general semaphores using binary semaphores lights.
5. Consider the following code fragment

```
int x = 10;
sem s1 = 1;
sem s2 = 0;
{ P(s1); P(s2); x = x-5; V(s2); Vs(1); } ||
{ P(s1); x = x*2; V(s1); V(s2); }
```

where sem denotes the semaphore type, $\parallel$ denotes the parallel composition while P and V are the operations on the semaphores we have seen. Say what are the possible final values for the variable x after execution of the fragment.

6. Provide a solution to the problem of dining philosophers using a centralized controller.

7. Extend the client-server application of the example 14.5.2 to the case where the number of clients is not statically known (use appropriate primitives that allow the creation of new names and the forwarding of names on channels).
8. Using any pseudo-language that allows only synchronous communication, simulate asynchronous communication by programming an appropriate process that realizes the asynchronous channel.
9. Write a program that solves the *problem of the unisex bathroom*: a bathroom can be used by both men and women, and an arbitrary number of people can be in the bathroom at the same time, however, there cannot be men and women at the same time.
10. Provide a solution to the dining philosophers' problem that uses a monitor. The monitor should have two operations `acquireForks( id )` and `releaseForks( id )`, of obvious meaning, where `id` is the identifier of the philosopher process that calls the operation.
11. Let a monitor be defined to operate a common account. The operations of the monitor must allow the deposit and withdrawal of money by more than one person (process) sharing the use of the account. The account balance cannot become negative, and if a withdrawal of an amount greater than the balance is requested, this operation must be blocked until the balance becomes greater than or equal to the requested amount.
12. Using asynchronous communication, develop filter processes to sort an array of n elements as follows. The various filter processes are cascaded one after the other. The first receives the data, one at a time, retains the minimum value and passes the remainder to the second process. The other processes do the same: they receive a stream of values from the previous one, retain the minimum and pass the remaining values to the next. Assume that each process can store only two values (the minimum and the next input data).
13. Produce code that, using asynchronous communication, solves the following problem of *stable marriage*. You have two vectors of n elements, A and B , respectively. Each member of A sorts any member of B from 1 to n (based on his preferences) and each member of B does the same with the members of A . A pairing is a two-way correspondence between A and B . A pairing is stable if there are no two pairs (a_1, b_1) and (a_2, b_2) in it such that a_1 prefers b_2 to b_1 and b_2 prefers a_1 to a_2 (i.e., there is no A-B pair in which both people prefer the other to their current partner).

References

1. C.A.R. Hoare, Communicating sequential processes. *Commun. ACM* **21**(8), 666–677 (1978)
2. R. Milner, J. Parrow, D. Walker, A calculus of mobile processes, I and II. *Inf. Comput.* **100**(1), 1–77 (1992)
3. E.W. Dijkstra, Co-operating sequential processes, in *Programming Languages*, ed. by F. Genuys (Academic-Press, London, 1968), pp. 43–112
4. E.W. Dijkstra, Hierarchical ordering of sequential processes. *Acta Informatica* **1**, 115–138 (1971)

5. E.W. Dijkstra, Guarded commands, non-determinacy and a calculus for the derivation of programs, in ed. by F.L. Bauer, K. Samelson, *Language Hierarchies and Interfaces*, volume 46 of *Lecture Notes in Computer Science* (Springer, 1975), pp. 111–124
6. G.R. Andrews, *Foundations of Multithreaded, Parallel, and Distributed Programming* (Addison-Wesley, 1999)
7. K. Arnold, J. Gosling, D. Holmes, *The Java Programming Language*, 4th edn. (Addison-Wesley Professional, Boston, 2005)
8. D. Lea, *Concurrent Programming in Java: Design Principles and Pattern*, 2nd ed. (Prentice Hall, 1999)